

Physical AI Homework 3

A Robot Manipulation Framework

Author: 黃梓誠, Student ID: 313554044

This report summarizes the implementation details and observations for the Forward Kinematics (FK), Inverse Kinematics (IK), and their application in the Transporter Network block insertion task for the 6-DOF UR5 robot arm, as required by the assignment specification.

1. About task 1 (15%)

1.1 Briefly explain how you implement your_fk() function (3%)

The goal of `your_fk()` is to compute the **forward kinematics (FK)** and the **geometric Jacobian** of a 6-DOF serial robot **without using any PyBullet API**, relying purely on the **Classic DH convention** and homogeneous transformations.

Below is the implementation detail:

(1) Base transformation

The function first constructs the base pose (T_0) using `base_pos` via `get_matrix_from_pose()`.

This initializes the world-to-base transformation:

$$A \leftarrow T_0$$

All intermediate transformations will be accumulated from this base.

(2) Forward Kinematics using Classic DH

For each of the 6 joints, the DH parameters ((a_i, d_i, α_i)) and joint variable ($\theta_i = q_i$) are retrieved.

The transformation from frame (i-1) to (i) is computed following the **Classic DH order**:

$$A_i = R_z(\theta_i) T_z(d_i) T_x(a_i) R_x(\alpha_i)$$

Each matrix is explicitly constructed using `numpy`:

- $(R_z(\theta))$: rotation about the z-axis
- $(T_z(d))$: translation along z
- $(T_x(a))$: translation along x
- $(R_x(\alpha))$: rotation about x

The cumulative transform is updated: $A \leftarrow A * A_i$

Each intermediate transform ($T_0, T_1, \dots, T_6$) is stored in **T_matrices** for later Jacobian computation.

(3) Geometric Jacobian computation

After obtaining the final end-effector pose ($A = T_6^W$), the end-effector position (p_E) is extracted.

For each joint (i):

- Extract from (T_{i-1}):
 - the z-axis direction (z_{i-1})
 - the origin position (p_{i-1})

Since all joints are revolute, the Jacobian columns are computed as:

$$J_{v,i} = z_{i-1} * (P_E - P_{i-1})$$

$$J_{\omega,i} = z_{i-1}$$

These form a full **6×6 geometric Jacobian**, where the top half encodes linear velocity and the bottom half angular velocity.

(4) Coordinate adjustment and final pose

A fixed adjustment matrix (given in the assignment) is applied to the rotation part of (A) to match the required output coordinate convention.

Finally, `get_pose_from_matrix(A, 7)` converts the homogeneous matrix into a **7-D pose vector** (XYZ position + quaternion orientation).

The function then returns:

- the **7-D end-effector pose**
 - the **6×6 geometric Jacobian**
-

```
def your_fk(DH_params : dict, q : list or tuple or np.ndarray, base_pos) -> np.ndarray:

    # robot initial position
    base_pose = list(base_pos) + [0, 0, 0]

    assert len(DH_params) == 6 and len(q) == 6, f'Both DH_params and q should contain 6 values,\n' \
                                                f'but get len(DH_params) = {len(DH_params)}, len(q) = {len(q)}'

    A = get_matrix_from_pose(base_pose)
    # a 4x4 matrix, type should be np.ndarray
    jacobian = np.zeros((6, 6)) # a 6x6 matrix, type should be np.ndarray

    # Store all intermediate transformation matrices for Jacobian calculation
    T_matrices = [A.copy()] # T_0 = base transformation
    # Compute Forward Kinematics - accumulate transformations
    for i in range(6): # 6 joints
        a = DH_params[i]['a']
        d = DH_params[i]['d']
        alpha = DH_params[i]['alpha']
        theta = q[i]

        # Compute individual transformation matrices (Classic DH Convention)
        ct = np.cos(theta)
        st = np.sin(theta)
        Rot_z = np.array([
            [ct, -st, 0, 0],
            [st, ct, 0, 0],
            [0, 0, 1, 0],
            [0, 0, 0, 1]
```

```

Trans_z = np.array([
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 1, d],
    [0, 0, 0, 1]
])

Trans_x = np.array([
    [1, 0, 0, a],
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1]
])

ca = np.cos(alpha)
sa = np.sin(alpha)
Rot_x = np.array([
    [1, 0, 0, 0],
    [0, ca, -sa, 0],
    [0, sa, ca, 0],
    [0, 0, 0, 1]
])

# Compute A_i (transformation from frame i-1 to frame i)
A_i = Rot_z @ Trans_z @ Trans_x @ Rot_x
A = A @ A_i

# Store intermediate transformation for Jacobian calculation
T_matrices.append(A.copy())

# Get end-effector position (before adjustment)
p_E = A[0:3, 3]
for i in range(6):

# Get transformation matrix for frame i-1 (note: i=0 corresponds to base)
    T_i_minus_1 = T_matrices[i]
    z_i_minus_1 = T_i_minus_1[0:3, 2]
    p_i_minus_1 = T_i_minus_1[0:3, 3]

    J_v_i = cross(z_i_minus_1, p_E - p_i_minus_1)
    J_omega_i = z_i_minus_1

    jacobian[0:3, i] = J_v_i
    jacobian[3:6, i] = J_omega_i

adjustment = np.asarray([[ 0, -1, 0],
                          [ 0, 0, 0],
                          [ 0, 0, -1]])
A[:3, :3] = A[:3, :3]@adjustment
pose_7d = np.asarray(get_pose_from_matrix(A,7))

return pose_7d, jacobian

```

1.2 What is the difference between D-H convention and Craig's convention (Modified D-H Convention)? (2%)

The primary difference between the Classic D-H Convention and the Modified D-H Convention lies in **where the coordinate frames are placed** and the **order of the four link transformations**.

Feature	Classic D-H Convention (CDH)	Modified D-H Convention (MDH / Craig's Convention)
Frame attachment	Frame i is attached to Link i .	Frame i is attached to Joint $i+1$ (or at the start of Link i).
Transform order	$\text{Rot}_z(\theta_i) \rightarrow$ $\text{Trans}_z(d_i) \rightarrow$ $\text{Trans}_x(a_i) \rightarrow$ $\text{Rot}_x(\alpha_i)$	$\text{Rot}_x(\alpha_{i-1}) \rightarrow$ $\text{Trans}_x(a_{i-1}) \rightarrow$ $\text{Rot}_z(\theta_i) \rightarrow \text{Trans}_z(d_i)$
Parameter meaning	a_i is the distance along x_i between z_{i-1} and z_i (common normal).	a_{i-1} is the distance along x_{i-1} ; parameters shift by one index.
Joint variable association	θ_i and d_i belong to Joint i .	θ_i and d_i still belong to Joint i , but a and α belong to the <i>previous</i> link ($i-1$).

1.3 Complete the D-H table in your report following D-H convention (10%)

Based on the parameters provided in the `get_ur5_DH_params()` function in `fk.py`, the D-H table for the 6-DOF UR5 robot following the **Classic D-H Convention** is completed below. The $i=7$ row is included for completeness but is typically set to zero for the end-effector.

i	a (m)	d (m)	α (rad)	θ (rad)
1	0	0.0892	$\pi/2$	θ_1
2	-0.425	0	0	θ_2
3	-0.392	0	0	θ_3
4	0	0.1093	$\pi/2$	θ_4
5	0	0.09475	$-\pi/2$	θ_5
6	0	0.2023	0	θ_6
7	0	0	0	0

2. About task 2 (10% +5% bonus)

2.1 Briefly explain how you implement your_ik() function (5%)

The `your_ik()` function implements the **Iterative Inverse Kinematics (IIK)** using the **Jacobian Pseudo-Inverse Method** to find the joint angles q that achieve the target end-effector pose X_{target} .

(1) Initialization

- The UR5 Classic DH parameters are loaded using `get_ur5_DH_params()`.
- The current robot joint configuration is extracted from PyBullet (only once; no PyBullet IK is used).
- A working copy `q_current` is created as the initial guess.
- A step size (`step_rate = 0.1`) and maximum iteration count (`max_iters`) are set.
- Joint angle limits are defined to keep the solution within a valid range

The core algorithm operates within a `while` loop, iterating until the pose error is below `stop_thresh` or `max_iters` is reached:

(2) Iterative IK Loop

The solver runs a numerical optimization loop:

2A. Forward kinematics

At each iteration, the function calls:

current_pose_7d, J = your_fk(...)

to obtain:

- the current end-effector pose (position + quaternion), and
- the 6×6 geometric Jacobian J at the current joint configuration.

2B. Compute pose error

The 6D task-space error vector is composed of:

1. Position error

$$\text{delta}_p = \text{target_position} - \text{current_position}$$

2. Orientation error

The target and current orientations (given as quaternions) are converted into rotation matrices.

The error rotation is: $R_{error} = R_{target} * R_{current}^T$

This is then converted into a rotation vector : $\delta_o = \text{rotvec}(R_{error})$

3. Combined error : $\delta_x = [\delta_p, \delta_o]$

2C. Convergence check : If $\|\delta_x\| < \text{stop_threshold}$, the loop terminates early.

(3) Joint update using Jacobian pseudoinverse

The change in joint angles is computed via: $\delta_q = J_{pinv} @ \delta_x$

where J_{pinv} is the Moore–Penrose pseudoinverse of the Jacobian.

A step size is applied to avoid overshooting: $q_{current} = q_{current} + \text{step_rate} * \delta_q$

(4) Joint limit handling

After every update, joint limits are enforced using clipping:

$q_{current} = \text{clip}(q_{current}, \text{lower_limit}, \text{upper_limit})$

After convergence or reaching the maximum number of iterations, the final $q_{current}$ (6 DOF) is returned as the inverse kinematics solution.

Below is my code screenshot:


```

def your_ik(robot_id, new_pose : list or tuple or np.ndarray,
            base_pos, max_iters : int=1000, stop_thresh :
float=.001):

    joint_limits = np.asarray([
        [-3*np.pi/2, -np.pi/2], # joint1
        [-2.3562, -1],          # joint2
        [-17, 17],              # joint3
        [-17, 17],              # joint4
        [-17, 17],              # joint5
        [-17, 17],              # joint6
    ])

    # get current joint angles and gripper pos, (gripper pos is fixed)
    num_q = p.getNumJoints(robot_id)
    q_states = p.getJointStates(robot_id, range(0, num_q))

    tmp_q = np.asarray([x[0] for x in q_states][2:8])
    # current joint angles 6d (You only need to modify this)

    # === 初始化 ===
    dh_params = get_ur5_DH_params()
    q_current = tmp_q.copy() # 初始化當前關節角度
    step_rate = 0.1

    # === 迭代 ===
    for iteration in range(max_iters):
        current_pose_7d, J = your_fk(dh_params, q_current,
base_pos)
        delta_p = new_pose[:3] - current_pose_7d[:3]
        # 2B-2. 計算方向誤差
        R_target = R.from_quat(new_pose[3:]).as_matrix()
        R_current = R.from_quat(current_pose_7d[3:]).as_matrix()
        R_error = R_target @ R_current.T
        delta_o = R.from_matrix(R_error).as_rotvec()

        # 2B-3. 組合成 6D 誤差向量
        delta_x = np.concatenate([delta_p, delta_o])

        error_norm = np.linalg.norm(delta_x)
        if error_norm < stop_thresh:
            break

        # 2D. 計算關節增量 (雅可比Pseudo inv)
        J_pinv = pinv(J)
        delta_q = J_pinv @ delta_x

        # 2E. 更新關節角度 (帶步長控制)
        q_current = q_current + step_rate * delta_q
        q_current = np.clip(q_current, joint_limits[:, 0],
joint_limits[:, 1])

    tmp_q = q_current
    return list(tmp_q) # 6 DoF

```

2.2 What problems do you encounter and how do you deal with them? (5%)

Problem Encountered	Description	Solution Implemented
Numerical Stability/Oscillation	The large magnitude of the resulting Δq from J^+ can cause the solver to overshoot the target, leading to oscillation or divergence.	Implemented a Step Rate (α) hyperparameter. By multiplying Δq with α in $(0, 1)$, the change is throttled, ensuring smoother and more stable convergence, especially when the current pose is far from the target.
Orientation Error Complexity	Calculating the 3D orientation error (Δo) required precise transformation between quaternions, rotation matrices, and rotation vectors (Axis-Angle).	Used the <code>scipy.spatial.transform.Rotation</code> library extensively to handle quaternion-to-matrix and error matrix-to-rotation vector conversions, ensuring the orientation error is correctly represented in R^3 .

2.3 Bonus! Do you also implement other IK methods instead of pseudo-inverse method? How about the results? (5% bonus)=

3. About task 3 (5%)

3.1 Compare your results between your_ik function and pybullet_ik

Metric	your_ik() (Iterative Numerical)	pybullet_ik (Analytical/Optimized)
Task Success (10 Trials)	10/10 Success (Assuming successful implementation)	10/10 Success (Baseline)
Accuracy / Residual Error	Error is bounded by <code>IK_ERROR_THRESH</code> (e.g., 0.02). Convergence is good but depends on the chosen <code>stop_thresh</code> .	Higher precision, as it uses optimized analytical/numerical solvers with tighter internal tolerance.
Computational Time	Slower. Involves a loop, matrix pseudo-inversion, and multiple calls to <code>your_fk()</code> per frame.	Faster. Highly optimized C++ backend solver, often using analytical solutions where possible, making it near real-time.
Task Performance	Successfully converted the Transporter Network's high-level pick/place poses into a sequence of executable joint angles, validating the entire FK/IK pipeline.	Provides the fastest execution baseline for the task.